

PART 9: UX, DESIGN & PRODUCT QUALITY (PM VIEW)

A PM who understands UX is unstoppable. A PM who doesn't is forever bottlenecked at the design lead's desk. You don't need to design pixels, but you need fluency: a vocabulary, a working knowledge of principles, the ability to critique work substantively, and the wisdom to know when to defer.

This chapter is your UX education for PMs. It's deeper than most PM curricula go because UX literacy is one of the highest-leverage skills you can build.

9.1 The PM's Role in UX

You are not the designer. You are not the design critic. You are not the pixel-pusher. So what *are* you?

You are:

1. **The user advocate in every meeting.** When eng says "let's hide this in settings to ship faster" or sales says "add this banner to drive conversion," you ask: "How will this affect the user's experience?" If no one asks, the user loses.
2. **The framer of UX problems.** "Users get confused on the pricing page" is a problem; "Make the pricing page modern" is a fashion request. The first is a UX brief; the second is a design vanity project.
3. **The integrator across surfaces.** Designers see one flow. You see how that flow connects to onboarding, to billing, to support, to email, to push, to web, to mobile. You ensure consistency across that whole.
4. **The judge of trade-offs the designer can't make alone.** Polish vs ship date. Power-user features vs novice clarity. Custom flow vs reused pattern. These have business implications; you bring those.
5. **The voice of "is this worth it?"** A 4-week beautiful redesign vs a 1-week functional update — when each makes sense is your call, in conversation with design.

You are not:

- Authority on visual aesthetics. (Your taste matters; the designer's expertise matters more.)
- The judge of design craft. (Designers critique designers; PMs critique whether designs solve user problems.)
- The owner of the design system or component library. (That's design.)

- The QA team. (Design review and product QA are different functions.)

When PMs overstep into design, they damage trust. When they understep, they ship inconsistent slop. The middle path is: own the *problem* and the *outcome*; defer on *form* unless form has product implications.

9.2 Nielsen's 10 Usability Heuristics — Deep Dive

Jakob Nielsen, co-founder of NN/g, published these in 1994. They remain the most-cited usability principles and the foundation of every PM's UX vocabulary. We'll go through each with examples and PM-relevant implications.

Heuristic 1: Visibility of System Status

Users should always be informed about what is happening through appropriate, timely feedback.

Examples:

- Loading spinners during long operations.
- "3 of 5 steps complete" progress bars.
- "Sent" / "Delivered" / "Read" indicators on chat apps.
- "Auto-saved at 3:42pm" in document editors.
- Optimistic UI: showing a sent message immediately, with a "sending" state until confirmed.

PM implication: When system status isn't visible, users assume the system is broken. The most common bug isn't actually broken behavior — it's hidden state. Every flow should answer the user's implicit question: "*Did that work?*" in under 2 seconds.

Anti-pattern: A button that does something but provides no feedback. Users click it 5 times, then assume it's broken, then either rage or churn.

Indian context: On low-bandwidth connections, network calls take 5–15 seconds. "*Is it loading or is the app dead?*" is the constant user question. Aggressive skeleton screens, optimistic UI, and explicit "Connecting..." states matter 10× more in India than in US-centric UX.

Heuristic 2: Match Between System and the Real World

The system should speak the user's language, with words, phrases, and concepts familiar to the user.

Examples:

- "Trash" instead of "Decommissioned files store."
- "Send money" instead of "Initiate fund transfer."
- A shopping cart icon for cart (matches the real-world metaphor).
- Date display: "March 15, 2025" or "15 March 2025" matched to user's locale (Americans vs Indians vs Europeans expect different formats).

PM implication: Engineering and product teams develop their own jargon. Users don't speak it. Audit your product's language. Replace internal-speak with user-speak. Examples to watch:

- "Dashboard" — does the user know what this means?
- "Workspace" — is this the project, the team, or the company?
- "Profile" vs "Account" — these are different in your system but identical to most users.

Indian context: Translation matters. "Submit" in English is fine. "जमा करें" (transliterated Hindi) sounds awkward to Indians who actually code-switch and prefer "Submit करें" (Hinglish). Building for Indian users without including bilingual UX experts is a major mistake.

Heuristic 3: User Control and Freedom

Users often perform actions by mistake. They need a clearly marked "emergency exit" to leave without going through an extended process.

Examples:

- Undo / Redo (the universal user-control mechanism).
- Cancel buttons in modals.
- Clear "Back" navigation that always works.
- Trash for deleted items, recoverable.
- Browser back button working as expected.

PM implication: The most user-friendly products have undo *everywhere*. The most user-hostile products force commitment at every step. When you make any destructive action, ask: "Can the user undo this?" If not, you need confirmation; if yes, no confirmation needed.

Anti-pattern: Multi-step wizards where you can't go back without losing input. The Indian government's tax filing portal is famously hostile here.

Anti-pattern: Confirmation dialogs that don't tell you what you're confirming ("Are you sure?" — sure of what?).

Heuristic 4: Consistency and Standards

Users should not have to wonder whether different words, situations, or actions mean the same thing. Follow platform and industry conventions.

Examples:

- Pull-to-refresh on mobile lists (universal mobile standard).
- Hamburger menu icon = navigation menu (now universal, even if it shouldn't have become so).
- Cmd+Z = undo on Mac; Ctrl+Z on Windows.
- Red = destructive action; green = positive action; blue = primary action (most cultures).

PM implication: When in doubt, follow the platform convention. iOS users expect iOS patterns; Android users expect Material Design patterns. Departing from convention requires real reason.

Internal consistency matters too: if your product has 3 different ways to delete a thing across 3 screens, you've created confusion. Audit periodically.

Anti-pattern: "Innovating" on standard interactions for the sake of it. The Snapchat UI famously confused users by refusing convention. Sometimes that's brand-defining; usually it's friction.

Heuristic 5: Error Prevention

Even better than good error messages is a careful design that prevents problems from occurring in the first place.

Examples:

- Disabling a "Submit" button until the form is valid.
- Real-time validation as the user types ("Username taken," "Password too short").
- Preventing destructive actions with confirmation only when the action is genuinely destructive (not for trivial things, where it becomes friction).
- "Are you sure you want to delete this?" only on items not recoverable from trash.
- Smart defaults that prevent errors (e.g., date pickers that only allow valid future dates).

PM implication: Most error messages signal a design failure earlier in the flow. If users keep submitting invalid forms, the form should make the right answer easier than the wrong one.

Worked example: A signup form requires "alphanumeric password, 8+ characters, 1 special character." If 30% of users hit the error, the design fails — change to a password meter that confirms strength as user types, with examples of what works.

Heuristic 6: Recognition Rather Than Recall

Minimize the user's memory load. Make objects, actions, and options visible.

Examples:

- Autocomplete and suggestions in search.
- Recently used files in the sidebar.
- Visible navigation rather than hidden command-line.
- Visible commands in menus rather than memorized keyboard shortcuts.
- "Continue where you left off" in video players, books, courses.

PM implication: Don't assume users remember anything. Show options. Save context. If users are typing in things they've typed before, you should be remembering and suggesting.

Anti-pattern: Slack's slash commands. Powerful for power users; invisible to everyone else. Recognition (visible buttons) outperforms recall (typed commands) for most users. Slack solved this with `/` triggering a menu — recognition layered on recall.

Heuristic 7: Flexibility and Efficiency of Use

Accelerators — invisible to novice users — may speed up interaction for expert users.

Examples:

- Keyboard shortcuts for power users (Ctrl+K for command palette in modern apps).
- Custom views, filters, saved searches.
- Macro/automation features (Notion's templates, Zapier-style flows).
- Bulk actions for users with many items.

PM implication: A great product has both. Don't pick one. Notion is the gold standard: novice users see a clean blank page; power users use slash commands and templates and Notion AI. Both groups feel the product is built for them.

Indian context: Apps that serve both metro power users and Tier-3 first-time users need to nail both. Most companies pick one and lose the other. Cred is metro-power-user-only; PhonePe successfully serves both.

Heuristic 8: Aesthetic and Minimalist Design

Interfaces should not contain information that is irrelevant or rarely needed. Every extra unit of information competes with the relevant.

Examples:

- Apple's traditional approach: hide complexity until needed.
- Stripe's checkout: only ask for what's strictly required.
- Google's homepage: a search bar, nothing else.

PM implication: Information density is a trade-off, not an absolute. B2B power-user tools often *should* be dense (Bloomberg Terminal, Excel, Linear) because experts process density faster than they wait for clean-but-slow interfaces. Consumer apps should be minimal because casual users get overwhelmed.

The wrong question: "Is this dense or minimal?" The right question: "Is the density appropriate for this user's expertise and frequency of use?"

Anti-pattern: Adding "tips" and "hints" everywhere because they seem helpful. They actually create noise.

Heuristic 9: Help Users Recognize, Diagnose, and Recover from Errors

Error messages should be in plain language, precisely indicate the problem, and constructively suggest a solution.

Examples of bad:

- "Error code 5023." (Why am I seeing this?)
- "Something went wrong." (Wrong how?)
- "Invalid input." (Which field?)

Examples of good:

- "Your card was declined. Try a different card or contact your bank."
- "We couldn't connect to the internet. Check your network and try again."
- "This email is already registered. [Sign in instead]"
- "Please enter a valid Indian PIN code (6 digits)."

PM implication: Audit error messages. Where they're cryptic, fix them. This is high-leverage cleanup — small effort, big user trust gain.

The "what now" rule: Every error should answer: *what just happened* and *what can I do about it*.

Heuristic 10: Help and Documentation

It may be necessary to provide help and documentation. Help should be easy to search, focused on the user's task, and concise.

Examples:

- In-app help search.
- Tooltips on hover for icons.
- Contextual hints during onboarding.
- Inline tutorials triggered by user state.
- Searchable help center for deeper questions.

PM implication: Help docs are a sign you couldn't make the UI self-explanatory. Strive for *no help docs needed* on common flows. But for complex products, great docs (Stripe's, Notion's, Linear's) are themselves a competitive moat.

Self-Audit: Score Your Product on the 10 Heuristics

Pick one product you work on or study. Score 1–5 on each heuristic. Find the lowest-scoring two. Those are your highest-leverage UX improvements.

9.3 Don Norman's Design Principles

Norman, in *The Design of Everyday Things* (1988, revised 2013), introduced concepts that PMs reference daily.

Affordances

What an object suggests it can do. A button affords "click." A door handle affords "pull." A flat plate on a door affords "push."

PM implication: Every interactive element should *look* like its function. A button should look clickable. A swipeable card should hint at swipe. Users shouldn't have to wonder "is this clickable?"

Anti-pattern: Flat design that strips visual cues. Modern flat-design UIs sometimes make it impossible to distinguish a label from a button. Material Design and iOS Human Interface Guidelines push back; consider their guidance.

Signifiers

Signs and signals that communicate affordances. A "Submit" label on a button is a signifier. An arrow icon on a swipe card is a signifier.

PM implication: Affordances alone aren't enough — users need explicit cues. Don't be afraid of labels and icons.

Mappings

The relationship between controls and what they control. A volume knob that turns clockwise to make sound louder has a natural mapping. A car's left/right blinker that maps to its turn direction has natural mapping.

PM implication: When designing controls, ask: does the layout/orientation match what it controls? An "advance" button on the right (for left-to-right readers) feels natural; placing it on the left feels wrong.

Feedback

The system tells the user what just happened. Tap a button → button visibly depresses → action confirmation appears. Without feedback, users repeat actions or assume failure.

PM implication: This overlaps with Heuristic #1 (visibility of status). Most "buggy" feeling apps actually work — they just don't give feedback.

Constraints

Limitations that prevent error. A USB plug shape constrains insertion to one direction (mostly). A date picker that only allows future dates constrains errors.

PM implication: Build constraints into the UI to prevent invalid states. Disable buttons when their action would be invalid. Filter pickers to only show valid options.

The Knowledge in the Head vs Knowledge in the World

Don Norman's framework: users have some knowledge in their heads (memorized) and rely on knowledge in the world (visible). Good design puts knowledge in the world.

PM implication: Recognition over recall (heuristic #6) is one expression of this. Show the answer; don't make users remember.

9.4 Information Architecture (IA)

IA is the organization of content, navigation, and labels so users can find what they need.

Card Sorting

A research method where users sort labels into categories to reveal mental models. Used to design navigation hierarchies. Tools: Optimal Workshop, Maze.

Tree Testing

After a navigation structure is proposed, give users a task ("find your billing settings") and see if they navigate through the tree successfully. Reveals where labels are confusing or hierarchies are wrong.

Common IA Patterns

- **Hierarchical:** Categories within categories (Amazon's catalog).
- **Faceted:** Multiple ways to filter the same set (e-commerce filters by price, brand, rating).
- **Hub-and-spoke:** A central home, radiating out to features (mobile apps).

- **Tag-based / network:** Items connect freely (Notion, Roam Research).

PM implication: Most navigation problems are IA problems, not visual design problems. Before redesigning the visuals, audit the structure. Often, "the menu is confusing" really means "the categories are wrong."

9.5 Common Interaction Design Patterns PMs Should Know

A vocabulary of standard UX patterns. Knowing these lets you communicate with designers fluently.

Patterns for navigation:

- Tab bar (mobile bottom navigation)
- Hamburger menu (sidebar trigger)
- Breadcrumbs (showing path within hierarchy)
- Back/forward (browser-style)
- Tabs (within a page)

Patterns for input:

- Forms (with validation)
- Multi-step wizards
- Inline editing
- Drag-and-drop
- Command palettes (Cmd+K)

Patterns for content:

- Lists (with filtering, sorting, pagination, infinite scroll)
- Cards
- Tables (especially in B2B)
- Carousels
- Galleries

Patterns for confirmation/feedback:

- Toasts (transient notifications)
- Modals (blocking dialogs)
- Inline alerts
- Empty states
- Loading skeletons

Patterns for delight:

- Microinteractions (button hover, swipe response)
- Animation (transitions between states)
- Haptic feedback (on mobile)
- Sound design (used carefully)
- Easter eggs (Slack's loading messages)

Patterns for AI products:

- Chat interfaces
- Inline suggestions (Copilot)
- Confidence indicators
- "Regenerate" / refinement
- "Why did you say this?" (explainability)
- Streaming text (vs waiting for full response)

A great PM can name and discuss these. They'll come up daily.

9.6 Accessibility (a11y)

Accessibility is the practice of designing products usable by people with disabilities — visual, auditory, motor, cognitive. It's not optional. It's the right thing to do, and increasingly the legal thing.

The WCAG Guidelines

The Web Content Accessibility Guidelines (WCAG) are the global standard. Three levels: A (basic), AA (most common compliance target), AAA (rarely required).

WCAG organizes around four principles (POUR):

- **Perceivable:** Users must be able to perceive the content (visual contrast, alt text, captions).
- **Operable:** Users must be able to operate the interface (keyboard navigation, sufficient time).
- **Understandable:** Information and operation must be understandable (clear language, predictable behavior).
- **Robust:** Content must work with assistive technologies (screen readers, voice control).

Practical Accessibility Checklist for PMs

For any product, check:

- ☐

Color contrast ratio $\geq 4.5:1$ for body text (use a tool like contrast-ratio.com or browser DevTools).

- ☐ All interactive elements reachable by keyboard (Tab, Enter, Esc).
- ☐ Focus indicators visible when tabbing through.
- ☐ Images have alt text.
- ☐ Forms have labels (not just placeholders).
- ☐ Videos have captions; audio has transcripts.
- ☐ No information conveyed by color alone (color-blind users).
- ☐ Buttons and links have text or aria-labels (not just icons).
- ☐ Page structure uses semantic HTML (h1, h2, main, nav).
- ☐ Animations can be disabled (some users get motion sickness).
- ☐ Text can be resized to 200% without breaking layout.
- ☐ Time-out warnings allow users to extend.

Why Accessibility Is Strategic, Not Just Ethical

- **Legal exposure.** US ADA lawsuits against inaccessible web products have skyrocketed. India's Rights of Persons with Disabilities Act 2016 + 2024 amendments require accessibility for digital services. EU's Accessibility Act becomes enforceable in 2025.
- **Market size.** ~15% of the global population has some disability. That's 1.2B+ people. India alone has ~70M+ persons with disabilities (Census + WHO estimates).
- **Aging population.** Accessibility features (large text, voice control) help aging users — a fast-growing demographic.
- **Universal benefit.** Curb cuts (originally for wheelchairs) help parents with strollers, delivery workers, travelers with luggage. Accessible design helps everyone.

A great PM treats accessibility as a Day-1 requirement, not an end-of-project add-on. Adding accessibility late is 5–10× more expensive than building it in.

Common Indian Context Failures

- Hindi/regional language fonts often have poor accessibility metadata.
- Voice readers struggle with bilingual content (Hinglish).
- Government and banking apps in India consistently rank as the most inaccessible — a major opportunity for inclusive PMs.

- Low-end Android devices don't support all accessibility features uniformly; test on real devices.

9.7 Inclusive Design

Beyond accessibility (which addresses disabilities), inclusive design considers all forms of human diversity: age, language, culture, education, technical skill, geographic context.

The Microsoft Inclusive Design Principles

1. **Recognize exclusion.** Existing products exclude some users. Identify who and why.
2. **Solve for one, extend to many.** Design for an extreme case (e.g., a user with no internet); the solution often helps everyone (offline mode benefits commuters too).
3. **Learn from diversity.** People at the margins of design know things that designers don't.

Worked Example: Designing UPI for India

UPI succeeded partly because it was inclusively designed:

- **Low-bandwidth tolerant:** Works on 2G connections (most older Indians).
- **Multilingual:** Available in 11+ languages.
- **Voice-first option:** UPI 123Pay works without a smartphone for feature-phone users.
- **Phone-number based:** No need to remember bank account numbers (most rural users couldn't).
- **Free for users:** No transaction fees.

This is inclusive design done well. Compare to many global payment products that assume English, smartphones, fast internet, and bank-account literacy — and miss India's 200M+ feature-phone population entirely.

Practical Inclusive Design for PMs

- Test on low-end devices (Redmi A1, sub-₹10K phones).
- Test on slow connections (Chrome DevTools throttling, real 2G if possible).
- Test in target languages — not just translated, but reviewed by native users.
- Test with users from different age groups, especially 50+ in your target market.
- Test with users at different literacy levels for consumer products targeting mass markets.

9.8 Indian-Context UX

A dedicated section because most PM literature ignores this and Indian PMs need it.

Key Indian UX considerations:

Device reality:

- Median Indian smartphone: 3-4GB RAM, mid-range processor, 2-year-old Android version.
- Many users on devices below ₹15,000.
- Storage is constrained; users uninstall apps to make room.
- Implication: Keep app size <100MB if possible; lazy-load aggressively; minimize background processes.

Connectivity:

- 4G is dominant but flaky in many areas.
- 5G is rolling out but coverage is uneven.
- Wi-Fi at home is rare outside metros.
- Implication: Build for offline-first or graceful-degradation. Cache aggressively. Show clear states for "no connection."

Language:

- English literacy is ~10% functional, much less for "comfortable use."
- Hindi is the largest market, but Tamil, Telugu, Bengali, Marathi, Gujarati are huge regional markets each.
- Hinglish (mixed) is the most-used everyday language for urban millennials.
- Implication: Localization is not optional for mass-market consumer products.

Trust signals:

- Indian users want trust signals before paying or trusting an app: brand recognition, customer reviews, "many users have used this," explicit reassurance.
- Implication: Show trust badges, user counts, testimonials. Don't hide them as "promotional."

Payment behavior:

- COD remains common (40%+ of e-commerce orders in non-metro India).
- UPI dominates digital payments; cards distant second.
- Implication: Support multiple payment methods; show real-time UPI feedback; have COD option for non-digital-trusting users.

Family / community decision-making:

- Major purchases (insurance, investments, big-ticket items) are often family decisions, not individual.
- Implication: Make sharing easy. "Save and discuss with family" is a real feature.

Vernacular content and voice:

- Voice search is growing fast in Indian markets.
- Vernacular content consumption (YouTube, JioCinema, ShareChat) is enormous.
- Implication: Voice input as a first-class option, especially for older users; vernacular content discovery prominent.

Indian PM UX Mistakes (common)

- Designing for English-speaking metro power users; missing the 90%.
- Optimizing for high-end devices; testing only on iPhones.
- Assuming Western trust signals work (5-star ratings, US-style testimonials) when local cues (verified by friends, recommended by bank, government-approved) work better.
- Hiding COD or making it second-class; alienating Tier 2/3 users.
- Imposing US-style minimalism that confuses Indian users used to information-dense interfaces (Indian portals, news sites, government sites all run on density).

9.9 Usability Testing for PMs

Usability testing is watching real users try to complete tasks with your product. It's the cheapest way to find UX problems.

Why PMs Should Do Their Own Usability Tests

You can hire UX researchers. But running 5 simple tests yourself will teach you more than reading 50 reports. The visceral experience of watching a user struggle with something you designed is unforgettable; it changes how you think.

Method: 5-User Usability Test

Per Nielsen, 5 users find ~85% of usability problems. Six is overkill; four is undershooting. Here's the protocol:

Recruit 5 representative users.

- Fit your target persona.
- Mix expertise (some new, some existing users).
- Pay them for their time (₹500-₹2,000 in India is fair, depending on context).

Define 3-5 tasks to test.

- Each task should have a clear success criterion.
- Examples: "Find and update your billing address" / "Sign up and complete your first transaction" / "Cancel your subscription."

Run the session (30-45 min):

1. **Warmup (5 min).** Build rapport. Ask about their typical use of similar products.
2. **Task introduction.** Read the task aloud, give them a card with the task written.
3. **Think-aloud protocol.** Ask them to narrate what they're thinking as they try.
4. **Don't help unless they're truly stuck.** When they ask "is this what I'm supposed to click?" the answer is "what do you think?"
5. **Note where they hesitate, what they misclick, what they say.** Don't help.
6. **After each task,** ask: "What was easy? What was hard?"
7. **Debrief (5 min).** Open feedback.

Synthesize:

- After 5 sessions, list all the problems observed.
- Group by frequency (how many users hit each) and severity (how badly it impedes the task).
- The top 3-5 are your priority fixes.

Tools

- **In-person:** A laptop and a notebook.
- **Remote:** Zoom + screen share. Or specialized tools like UserTesting, Maze, Lookback (lookback.com).
- **Recording:** With consent, record. The recording is gold for sharing with team.

Common Mistakes

- Leading users ("Now click the blue button"). Don't.
- Testing with friends/colleagues. They'll be too kind. Recruit real strangers.
- Asking "do you like this?" Like is irrelevant; usability is what matters.
- Skipping the synthesis. The notes must be turned into actionable insights.
- Not sharing findings widely. The whole team should watch a few sessions; their empathy increases.

9.10 Quality Trade-offs: Speed vs Craft vs Delight

Every product team navigates this triangle. There's no universal right answer.

The Three Modes

Speed mode. Ship fast, iterate. Beauty is secondary; learning is primary.

- Best for: 0→1 stage; competitive races; experimental features.

- Example: Snapchat's early features shipped weekly with rough edges. They were learning.

Craft mode. Ship slower, polish more. Make it beautiful and reliable.

- Best for: Mature products in competitive UX wars; flagship features; brand-defining moments.
- Example: Apple's iOS releases — annual, polished, considered.

Delight mode. Ship something users will love and remember.

- Best for: Differentiation; word-of-mouth-driven products; premium positioning.
- Example: Notion's animations. Linear's smooth interactions. Loom's playful brand voice.

When to Choose What

Stage / Context	Speed	Craft	Delight
Pre-PMF	★★★	★	★
Early growth	★★★	★★	★
Scaling	★★	★★★	★★
Mature, defending	★	★★★	★★★
Premium positioning	★	★★★	★★★
Mass market consumer	★★	★★	★★★
Enterprise B2B	★★	★★★	★
AI / experimental	★★★	★	★★

This is rough. The right answer depends heavily on your specific competitive position.

Avoiding the False Trade-off

Sometimes "we don't have time for craft" is a copout. Designers and engineers can usually find clever ways to ship fast *and* well — if pushed to. The PM's role: don't accept "we have to ship rough" as the default. Push for the smaller-but-polished version where possible.

9.11 UX Critique Skill

A great PM can look at a screen and articulate, in 60 seconds, what's working and what's not. This is a learnable skill.

The 60-Second UX Critique Framework

For any screen:

1. **What's the primary user goal here?** (If you can't tell in 5 seconds, the screen has lost.)

2. **Where does my eye go first?** (Should match the primary goal.)
3. **What's the dominant action?** (Should be obvious.)
4. **What's missing?** (Often: status, feedback, error states.)
5. **What's extra?** (Often: rare-use elements crowding common-use elements.)
6. **What would a confused user do?** (The screen should help, not panic them.)
7. **How does this connect to the next/previous screen?** (Continuity matters.)

Example: Critiquing a Hypothetical Banking App Home Screen

Observation: Top of screen has bank logo + notification icon; below, 6 colored tiles in a 2x3 grid for "Pay," "Send," "Cards," "Accounts," "Investments," "Loans"; below, a horizontal carousel of promotions; bottom tab bar has 5 icons with text labels.

What's working:

- Tab bar follows iOS/Android convention (heuristic #4).
- Tiles use color and icons together (not relying on color alone — accessibility win).
- Primary actions are above the fold.

What's not:

- All 6 tiles look equally important — no visual hierarchy. Most users want "Pay" most often; it should be the largest tile or in the prime upper-left position.
- The promotions carousel pushes secondary content above primary actions for scrolling users on smaller screens.
- No "recent activity" or "balance" — users open banking apps to see "where do I stand?" not "what new things are there?"
- Notification icon position competes with the brand logo for attention.

This is structured, specific, and actionable critique. It's the level you should be able to do in any product review.

9.12 Working with Designers — Advanced

We covered the basics in Part 6. Here's the deeper craft.

Co-creating from problem to solution

The best PM-designer collaborations don't have a "PRD handoff." They have:

1. **Joint problem framing (1 week):** PM brings research; designer brings frame.
2. **Divergent exploration (1 week):** Designer explores 3+ directions; PM gives feedback on which addresses the problem best.

3. **Convergent design (1-2 weeks):** Selected direction refined; PM shapes flow and edge cases.
4. **Iteration with users (1 week):** Tested with real users; revised.
5. **Engineering handoff:** Joint by PM and designer.

This collaboration produces 2–3× better outcomes than the PM-writes-PRD-designer-makes-screens model.

Critique vs Direction

When reviewing design, PMs should:

Give critique (welcome):

- "This flow seems to assume the user knows X. What if they don't?"
- "I'm worried about users on small screens — does this layout still work?"
- "What's the empty state look like?"
- "How does this handle [error case]?"

Avoid direction (not welcome):

- "Move that button left."
- "Make the heading bigger."
- "Use a different color."

The first set is "I'm worried about user outcomes." The second is "I want to design." Your designer will love the first and resent the second.

When Design and Product Disagree

Sometimes designers and PMs have genuinely different views. The disagreement is usually about:

- **Scope:** Designer wants to redesign X; PM thinks scope is just Y. Resolution: clarify the user problem and whether X is needed to solve it.
- **Polish vs ship:** Designer wants more iteration; PM wants to ship. Resolution: specify what's "good enough"; what's must-have vs nice-to-have polish.
- **Power user vs mass user:** Designer optimizes for one; PM advocates for another. Resolution: data on user mix; sometimes both are right and the answer is "two flows."

In conflict, the PM's role is to bring the user data and the strategic frame. The designer's role is to bring craft expertise. Decide together; if you can't, escalate to design lead + PM lead together. Don't let it fester.

9.13 Design Systems

A design system is a shared library of components, patterns, tokens (colors, spacing, typography), and rules. Examples: Material Design (Google), Human Interface Guidelines (Apple), Polaris (Shopify), Atlassian Design System.

Why Design Systems Matter for PMs

- **Consistency.** Without a design system, every team designs slightly differently. The product feels disjointed.
- **Speed.** Components are pre-built; designers and engineers don't reinvent buttons.
- **Quality.** Components are tested for accessibility, edge cases, dark mode.
- **Brand integrity.** A design system is the operational expression of brand.

PM Implications

- When you propose a feature, ask: does this fit our design system? If not, why not?
- Avoid creating new components when existing ones suffice. Custom components are expensive and degrade consistency.
- Recognize that some product needs may justify extending the system — but coordinate, don't ad-hoc.
- Design systems need maintenance. Budget for it.

9.14 Animation, Microinteractions, and Emotional Design

The difference between functional and beloved is often emotional.

Microinteractions

Tiny moments where the product responds to user action: a button hover, a swipe response, a "like" animation, a successful save confirmation. Each is small; together they create *feel*.

Examples of beloved microinteractions:

- iOS pull-to-refresh with rubber-band physics.
- Linear's animations between views.
- Stripe's checkout button transformation.
- WhatsApp's two ticks turning blue.
- Notion's drag-and-drop.

PM implication: Microinteractions cost engineering time. They're often de-prioritized. A great PM identifies the *3-5 moments* in their product that deserve microinteraction investment, and protects budget for them.

Animation as Communication

Animation isn't decoration; it's communication. A modal that slides up tells the user "I'm temporary, dismiss me by sliding back down." A spinning loader tells the user "I'm working." A subtle bounce on a button confirms "yes, I noticed your tap."

Animation rules:

- Fast (150–300ms) for everyday actions.
- Easing curves (not linear) — feels more natural.
- Disable for users who request reduced motion (accessibility).
- Don't animate just because you can.

Emotional Design

Don Norman's *Emotional Design* identifies three levels:

- **Visceral:** Immediate gut reaction to looks/feel.
- **Behavioral:** Pleasure of use, when interactions feel smooth.
- **Reflective:** Long-term meaning — does this product reflect who I am?

Apple, Tesla, Notion, and Cred (Indian) succeed largely on reflective design — using their products signals identity. PMs at premium-positioned products should think about all three layers.

9.15 Dark Patterns and PM Ethics

Dark patterns are UX designs intended to manipulate users against their interest. They are everywhere. Avoid them; advocate against them; they damage long-term trust even when they win short-term metrics.

Common Dark Patterns

- **Roach motel:** Easy to sign up; impossible to cancel. (Many gym memberships, some streaming services.)
- **Confirmshaming:** "No thanks, I don't want to save money." Manipulative consent.
- **Forced continuity:** Free trial that auto-bills without warning.
- **Hidden costs:** Final checkout reveals fees not disclosed earlier.
- **Bait and switch:** Click "X" to close ad, but "X" actually starts the action.
- **Misdirection:** Big colorful "Continue" button leads to upsell; tiny grey "Skip" link leads to actual continue.
- **Privacy zuckering:** Tricking users into sharing more than they intend.
- **Disguised ads:** Ads that look like content.

Why PMs Should Refuse Dark Patterns

- They erode trust permanently.
- Increasing regulation (EU Digital Services Act, India's DPDP Act 2023) makes them legal liabilities.
- They optimize for short-term conversion at the cost of LTV.
- They contradict the stated purpose of most companies' "user-centered" claims.
- They feel bad to work on. Engineering and design teams often resent shipping them; morale suffers.

If asked to ship a dark pattern:

1. Articulate the long-term cost. Often leadership doesn't realize.
2. Propose a non-dark alternative. There's almost always one.
3. If forced anyway, document the decision in writing for future audit.
4. If repeatedly asked to compromise ethics: the right answer is sometimes to leave.

Ethical PM Principles

- **Default to user interest when in doubt.**
- **Don't engineer addiction.** Engagement is good; manipulative addiction (slot-machine dynamics, infinite-scroll dopamine loops on harmful content) is not.
- **Respect informed consent.** Users should know what they're signing up for.
- **Protect vulnerable users.** Children, the elderly, those in distress need extra care.
- **Be skeptical of "growth at all costs."** Sometimes the costs are real, just hidden in the future.

For deeper reading: *Ruined by Design* by Mike Monteiro, the *Tristan Harris / Center for Humane Technology* movement, *Indistractable* by Nir Eyal (a course-correction by the *Hooked* author).

9.16 Practice Exercises

Exercise 9.1 — Heuristic Audit

Pick a product (any app or website). Score it 1–5 on each of Nielsen's 10 heuristics. Identify the 3 lowest scores. For each, propose one specific improvement.

Exercise 9.2 — Critique an Indian Banking App

Open any major Indian banking app (HDFC, SBI, ICICI). Spend 10 minutes using it for a real task (e.g., transferring money). Write a 1-page critique covering: navigation, language, accessibility, error handling, trust signals, and one specific improvement.

Exercise 9.3 — Accessibility Test

Take any app you work on or use frequently. Run through the a11y checklist in 9.6. Identify gaps. Estimate effort to fix the top 3.

Exercise 9.4 — Design System Audit

Pick a product you use. Identify how many distinct button styles you can spot. Comment on the consistency (or lack of).

Exercise 9.5 — UX for Indian Tier-3 Users

Design a hypothetical "first-time mutual fund investment" flow for a Tier-3 Indian user (40-year-old shopkeeper, Hindi-first, ₹15K Android, low English literacy). Sketch the flow. Identify 5 design decisions you'd make differently than a metro flow.

Exercise 9.6 — Dark Pattern Spotting

Find a product you use that exhibits at least one dark pattern. Document it. Propose a non-dark alternative that achieves the same business goal.

Exercise 9.7 — Critique an AI Product UX

Use any AI chat product (ChatGPT, Claude, Perplexity). Identify 3 UX choices you find well-designed and 3 you'd improve. Be specific.

Exercise 9.8 — Write a Design Brief

For a feature you'd like to build, write a design brief (not a PRD — focused on UX problem only). Include: user goal, current friction, success criteria, and constraints. ~1 page.

Solutions

Solution 9.5 — Tier-3 mutual fund flow design decisions:

1. **Language: Default to Hindi (or detected regional).** Don't ask users to switch. Detect from device locale or first interaction.
2. **No financial jargon.** Replace "AUM," "NAV," "expense ratio" with plain language: "kitna paisa lagana hai" (how much to invest), "kitna milega" (estimated returns).
3. **Voice-driven onboarding.** A narrator reads each step aloud. Don't assume reading.
4. **Trust signals up front.** "RBI/SEBI registered," "Bank XXX partner" — visible from first screen.
5. **Smaller numerical defaults.** Default investment amount: ₹500, not ₹5,000. Build comfort with small commitments.
6. **No timer / urgency tactics.** Indian users in Tier 3 deeply distrust "act now" pressure. Calm, deliberate flow.

7. **Family-share built in.** "Discuss with family" button to share details via WhatsApp before committing.
8. **Pause-able flow.** Save state and resume — Indian users will often pause to consult.
9. **Upfront on risks in plain language.** "Paisa kam ho sakta hai" (money can decrease) — direct, not buried in legal text.
10. **Multiple entry/exit points** — not a single linear funnel, since users may explore, leave, return.

These decisions reflect Indian Tier-3 reality. Most fintech apps in India still don't do them well — there's a real opportunity for inclusively-designed products.

9.17 AI Leverage in UX

Use case 1: Heuristic critique on screenshots.

Prompt: "I'm pasting a screenshot of [feature]. Critique it against Nielsen's 10 usability heuristics. For each heuristic where it fails, suggest a specific improvement."

Multimodal AI can do this surprisingly well. Verify with your own judgment.

Use case 2: Generating empty states, error messages, microcopy.

Prompt: "Generate 5 friendly, on-brand error messages for: 'user tried to upload a file that's too large.' Brand voice: [warm/professional/playful]. 50 character limit. Include a 'what next' action."

Use case 3: Accessibility audit.

Prompt: "For this UI [paste HTML/description], do an accessibility review. Flag: missing alt text, poor contrast, missing labels, keyboard-trap risks, and other WCAG AA issues. Suggest fixes."

Use case 4: Vernacular translation review.

Prompt: "I have these strings in English [paste]. Translate to Hindi and Tamil. Then critique each translation: does it sound natural? Is the tone appropriate (formal/casual)? Are there cultural mismatches?"

Use case 5: User flow diagramming.

Prompt: "Sketch a user flow (in text/Mermaid syntax) for: 'first-time user signs up and completes their first investment.' Include happy path, common error states, and abandonment recovery."

Use case 6: Inclusive design check.

Prompt: "For the feature [describe], who might be excluded by current design assumptions? Consider: age, language, literacy, device, connectivity, disability, geography. For each, suggest one design change to include them."

9.18 References for Part 9

Books:

- *The Design of Everyday Things* — Don Norman. Foundational. ([Amazon](#))
- *Don't Make Me Think* — Steve Krug. The web usability classic. ([Amazon](#))
- *About Face* — Alan Cooper. Comprehensive interaction design. ([Amazon](#))
- *100 Things Every Designer Needs to Know About People* — Susan Weinschenk. Behavioral psychology for UX. ([Amazon](#))
- *Microinteractions* — Dan Saffer. The art of small interactions. ([Amazon](#))
- *Designing for Behavior Change* — Stephen Wendel. Applied behavioral design. ([Amazon](#))
- *Refactoring UI* — Adam Wathan & Steve Schoger. Practical visual design for non-designers. ([refactoringui.com](#))
- *Inclusive Design Patterns* — Heydon Pickering. Practical accessibility. ([smashingmagazine.com](#))
- *Ruined by Design* — Mike Monteiro. Ethics-focused. ([Amazon](#))
- *A Web for Everyone* — Sarah Horton & Whitney Quesenbery. Inclusive design. ([Amazon](#))

Web resources:

- Nielsen Norman Group — [nngroup.com](#). Encyclopedic UX research.
- Smashing Magazine — [smashingmagazine.com](#).
- Material Design — [m3.material.io](#). Google's design system, free reference.
- Apple Human Interface Guidelines — [developer.apple.com/design](#).
- Atlassian Design System — [atlassian.design](#). One of the best public design systems.
- Shopify Polaris — [polaris.shopify.com](#).
- WCAG 2.2 — [w3.org/TR/WCAG22](#).
- A11y Project — [a11yproject.com](#). Practical accessibility checklist.
- Refactoring UI's free articles. ([refactoringui.com](#))
- Growth.Design — [growth.design](#). Visual UX case studies.

Tools:

- **Design tools:** Figma, Sketch.
- **Prototyping:** Figma, Framer, Principle.
- **Accessibility testing:** axe DevTools (browser extension), Wave, Lighthouse.
- **Color contrast:** contrast-ratio.com, Stark plugin for Figma.
- **User testing:** UserTesting, Maze, Lookback.
- **Card sorting / IA:** Optimal Workshop.
- **Session recording:** FullStory, Hotjar.

Indian-context UX:

- The Indian UX Studio podcast.
- "Designing for India" — Lollypop Design's case studies.
- Google's "Designing for India" guidelines (search Google Developers).
- IIT Bombay's Industrial Design Centre publications.

Daily practice:

- Critique 1 product/screen per day for 60 days. Use the framework in 9.11.
- Subscribe to Growth.Design weekly case studies.
- Take any major product redesign in the news; analyze what changed and why.
- Practice empathy walking — try using your product (or a competitor's) on a 2G connection, on a low-end device, in a language you don't speak.

End of Part 9. Up next: the people skills that turn good PMs into great ones.

\newpage